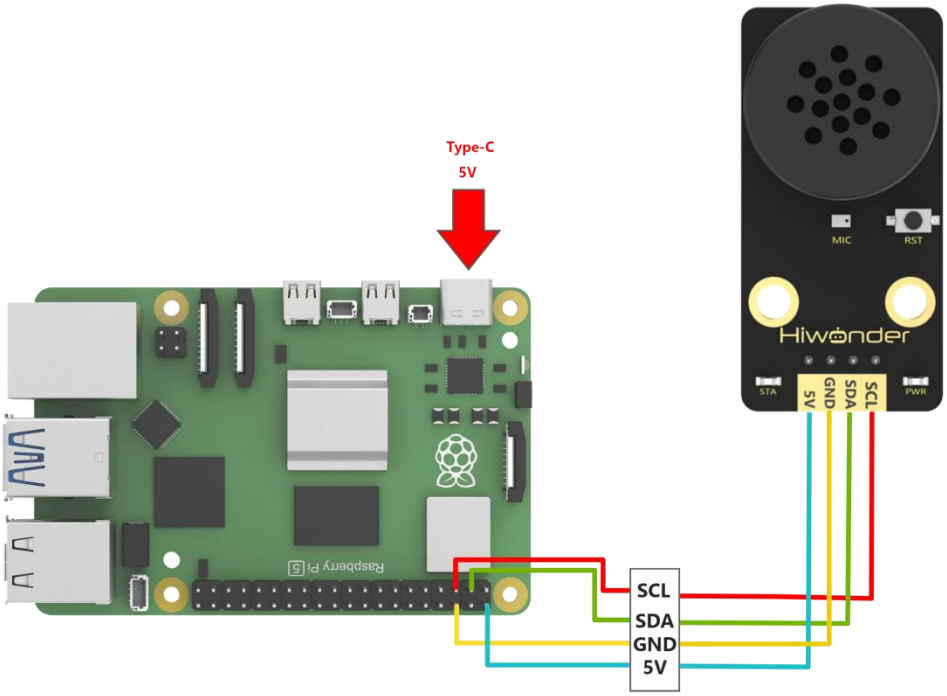
01 Raspberry Pi Voice Recognition

1. Preparation

1.1 Wiring Setup

When wiring, the 5V, GND, SDA, and SCL pins of the voice interaction module need to be connected to the Raspberry Pi. The wiring method is shown in the image below:



Note:

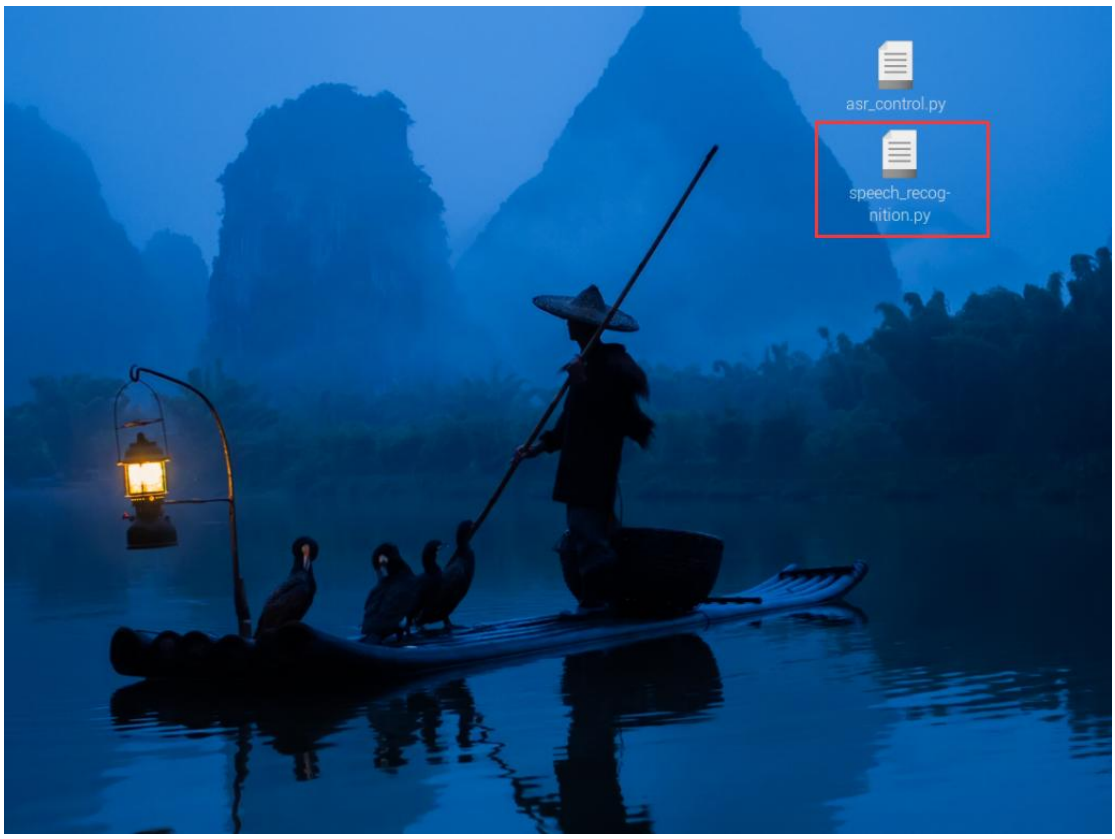
1. **Before** powering on, make sure no metal objects come into contact with the motherboard, as this could cause a short circuit from the pins on the underside, potentially damaging the board.

2. The I2C pin configuration on the Raspberry Pi 4 is identical to that on the **Raspberry** Pi 5, so this wiring method is also compatible with the Raspberry Pi 4.

1.2 Importing and Running the Program

Note: For instructions on installing and using MobaXterm and VNC Viewer, refer to "4.6.1 Remote Connectivity and File Transfer\01 Remote Connectivity and File Transfer."

1) Use MobaXterm to transfer the "speech_recognition.py" file from the "02 Program Files" folder (located in the same directory as this document) to the Raspberry Pi. In this example, we drag the file to the desktop. Then, connect to the Raspberry Pi using VNC Viewer.



2) On the Raspberry Pi, press **Ctrl + Alt + T** to open the terminal.

3) Navigate to the directory where the program is stored by entering:

```
cd Desktop/
```

```
❯ cd Desktop/
~/Desktop
```

4) Run the program with the following command:

python3 speech_recognition.py

```
❯ python3 speech_recognition.py
```

3. Test Case

This example demonstrates how the Raspberry Pi retrieves recognition results from the voice interaction module and displays them in the command-line terminal.

3.1 Expected Behavior

Note: Before issuing commands, you must wake up the module by saying **"Hello Hiwonder"**

1) When the voice interaction module detects **"Move forward"**, it responds with **"Moving forward"**, and the terminal outputs: go
2) When it detects **"Move backward"**, it responds with **"Moving backward"**, and the terminal outputs: back
3) When it detects **"Turn left"**, it responds with **"Turning left"**, and the terminal outputs: left
4) When it detects **"Turn right"**, it responds with **"Turning right"**, and the terminal outputs: right
5) When it detects **"Stop"** or **"Halt"**, it responds with **"Received"**, and the terminal outputs: stop

The terminal output should look similar to this:

```
❯ python3 speech_recognition.py
go
back
left
right
stop
```

3.2 Brief Program Analysis

1) Library Imports: The program imports the smbus library for I2C bus communication and the time library for time processing. These libraries facilitate communication with the voice interaction module.

```
#!/usr/bin/python3
# coding=utf8
import smbus
import time
```

2) Object Creation: An object named asr_module is instantiated from the ASRModule class to represent the voice interaction module. Another variable, recognition_result, stores the recognition output. During initialization, the I2C address of the integrated voice interaction module is provided to establish a communication link with the Raspberry Pi bus.

```
if __name__ == "__main__":
    asr_module = ASRModule(I2C_ADDR)
```

3) Class Initialization: When the ASRModule class is instantiated, it automatically creates a member object bus by instantiating the SMBus class from the smbus library. This enables communication with the Raspberry Pi's I2C bus. The provided I2C slave address is specified for communication during initialization.

```
class ASRModule:
    def __init__(self,address, bus=1):
        # 初始化 I2C 总线和设备地址
        self.bus = smbus.SMBus(bus)  # 使用 I2C 总线 1
        self.address = address  # 设备的 I2C 地址
        self.send = [0, 0]  # 初始化发送数据的列表
```

4) Reading Recognition Results: This section focuses on reading recognition result data stored in the module's registers. To achieve this, a function named wire_read_data_array is defined within the ASRModule class to read data from the slave device via the I2C bus.

The function accepts two parameters: the register address reg and the length of data length to be read.

It leverages the read_i2c_block_data() function from the bus (an SMBus object) to read data from the specified register.

The third parameter of read_i2c_block_data() specifies the number of bytes to read, returning a list with a length equal to the number of bytes read. This list requires no additional processing and can serve as the direct output of wire_read_data_array().

If bus communication fails and data cannot be read, an IOError exception is raised. In such cases, read_i2c_block_data() returns no value. To maintain consistent output, an empty list is returned in the exception handler.

```
    def wire_read_data_array(self, reg, length):
        """
        从指定寄存器读取字节列表
        :param reg: 寄存器地址
        :param length: 要读取的字节数
        :return: 读取到的字节列表，失败时返回空列表
        """
        try:
            result = self.bus.read_i2c_block_data(self.address, reg, length) # 从设备读取字节列表
            return result # 返回读取结果
        except IOError:
            return [] # 读取失败，返回空列表
```

5) Retrieving Recognition Results: The rec_recognition() function in the ASRModule class retrieves the recognized keyword results.

It calls wire_read_data_array() and checks whether the return value is an empty list.

If the list is empty, it indicates a read failure, prompting the function to return 0. Otherwise, it returns the recognition result.

```
def rec_recognition(self):
    """
    Read recognition result
    :return: The recognition result, returns 0 if reading fails
    """
    result = self.wire_read_data_array(ASR_RESULT_ADDR, 1) # Read a byte from the result register
    if result:
        return result # Return the read result
    return 0  # Return 0 if no result is read
```

6) Main Function Execution: After instantiating the voice interaction module class in the main function, the asr_module.rec_recognition() function is called repeatedly in a loop to continuously obtain data from the voice interaction module.

```
while True:
    recognition_result = asr_module.rec_recognition()
```

7) Recognition Results:

Recognizing the word "forward" returns 0x01.

Recognizing "backward" returns 0x02, and other keywords follow a similar pattern, with each keyword corresponding to a unique keyword ID. The Raspberry Pi prints the associated English meaning to the command line before moving to the next loop.

For the complete list of return values, refer to "5. Firmware Development > 5.1 Voice Chip Firmware Development".

If the user’s speech is not recognized, 0x00 is returned, causing the program to skip any action and proceed directly to the next loop.

```
        if recognition_result[0] != 0:
            if recognition_result[0] == 1:
                print("go")
            elif recognition_result[0] == 2:
                print("back")
            elif recognition_result[0] == 3:
                print("left")
            elif recognition_result[0] == 4:
                print("right")
            elif recognition_result[0] == 9:
                print("stop")
```